

Stability-Aware Predictive Kubernetes Scaling

Project Report

MSc Research Project / (Research) Practicum / Internship
Cloud Computing

Pooja
Student ID: TBD

School of Computing
National College of Ireland

Supervisor: TBD

September 2026

Stability-Aware Predictive Kubernetes Scaling

Pooja
TBD

Abstract

Wanigasooriya & Ekanayake (2026), “NimbusGuard: A Novel Framework for Proactive Kubernetes Autoscaling Using Deep Q-Networks” (IEEE ICOIN 2026), show that a DQN+LSTM proactive autoscaler beats reactive HPA and event-driven KEDA on responsiveness, but their own results table names the cost directly: NimbusGuard was “the most agile and least stable system,” with the highest average replica count and double the scaling events of the reactive baselines – a trade-off they flag as future work rather than resolve. This project reproduces that trade-off on a synthetic cyclical workload with an unfiltered proactive scaler (**AggressivePAKS**: a feed-forward regressor forecasting next-step demand, acting immediately on every prediction), confirming it clearly beats reactive HPA on SLA violations (11.0 vs. 20.0 across 5 seeds) at the cost of more scaling events (384.4 vs. 407.4, comparable) and higher over-provisioning (45.8% vs. 40.9%). A stability-aware controller (**StabilityAwarePAKS**) is then built on the same predictor, adding exponential smoothing and a hysteresis/cooldown rule before acting – the calmer design NimbusGuard names as future work but does not build. Across the same 5 seeds, it cuts scaling events by 57% (165.8 vs. 384.4) and pod-count volatility by 32% (1.95 vs. 2.86) relative to the aggressive baseline, while still beating reactive HPA on SLA violations (18.0 vs. 20.0) – though by less than the unfiltered aggressive policy does (11.0). The result is an honest trade-off, not a clean win on every axis: stability is recovered substantially, at a partial cost to the SLA-violation benefit that made the proactive approach attractive in the first place.

1 Introduction

Kubernetes’ built-in autoscalers – the Horizontal Pod Autoscaler (HPA) and the event-driven KEDA – are reactive: they scale only once a resource metric has already crossed a threshold, which under bursty, cyclical workloads causes either SLA-violating under-provisioning during the delay or costly over-provisioning afterward. Proactive, ML-based forecasting is a natural fix, but it introduces a new risk: acting on every noisy prediction can make the system more agile at the cost of being less stable.

Wanigasooriya & Ekanayake Wanigasooriya and Ekanayake (2026) demonstrate exactly this with NimbusGuard, a Deep Q-Network + LSTM proactive autoscaler evaluated against reactive HPA and KEDA on a real Kubernetes testbed. NimbusGuard wins on responsiveness, but their own results table shows it running with the highest average replica count and double the total scaling events of either reactive baseline – described in their own words as “the most agile and least stable system.” They name addressing this stability cost as future work.

Research question: can a proactive autoscaler keep most of its SLA-violation advantage over reactive HPA while substantially reducing the instability (scaling-event frequency, pod-count volatility) that an unfiltered, DQN-style proactive policy exhibits?

Objectives:

- Reproduce the trade-off NimbusGuard reports – proactive scaling beats reactive HPA on SLA compliance, at a cost in over-provisioning and instability – with a simpler, fully reproducible feed-forward predictor rather than a full DQN+LSTM+LLM pipeline.
- Design and build the stability fix NimbusGuard names but does not build: smoothing and hysteresis applied to the same underlying forecast.
- Quantify the trade-off honestly across multiple metrics and multiple random seeds, rather than reporting only the metrics that favour the new design.

The remainder of this report is structured as follows: Section 2 reviews the baseline paper and the gap it names; Section 4 describes the three scaling policies compared; Section 5 describes the workload simulator, training pipeline, and deployment; Section 6 reports results across 5 seeds; Section 7 discusses the trade-off and future work.

2 Related Work

Kubernetes’ built-in Horizontal Pod Autoscaler (HPA) and event-driven autoscalers like KEDA are both reactive: they scale only after a metric threshold is already crossed, which under bursty workloads leads to either SLA-violating under-provisioning or costly over-provisioning.

2.1 Baseline: Proactive Autoscaling with Deep Q-Networks

The baseline for this project is Wanigasooriya & Ekanayake Wanigasooriya and Ekanayake (2026), “NimbusGuard: A Novel Framework for Proactive Kubernetes Autoscaling Using Deep Q-Networks” (IEEE ICOIN 2026). NimbusGuard combines a Deep Q-Network scaling agent with an LSTM workload forecaster, evaluated against reactive HPA and event-driven KEDA on a real KinD Kubernetes testbed under a phased load pattern (ramp-up, sustained load, peak load, cooldown). Their results show NimbusGuard reduces the metric they care most about – responsiveness – outperforming both reactive baselines.

2.2 The Gap This Project Targets

The same results table exposes the cost of that responsiveness. NimbusGuard runs with the highest average replica count (5.44 vs. 3.05 for HPA and 2.93 for KEDA) and double the total scaling events of either reactive baseline (8 vs. 4). The paper states this directly: “NimbusGuard was the most agile and least stable system... these findings highlight a fundamental trade-off: the proactive, performance-focused scaling of NimbusGuard versus the reactive, cost-efficient stability of traditional autoscalers.” Their own future-work section gestures at addressing this (expanding the action space, hybridising with Vertical Pod Autoscaling) but does not build or evaluate a fix.

2.3 Smoothing and Hysteresis in Autoscaling

Damping noisy scaling signals with smoothing and cooldown windows is a long-established idea in autoscaler design generally (Kubernetes HPA itself has a stabilization-window option), but it is not what NimbusGuard’s own DQN+LSTM pipeline does – their agent acts on every raw forecast. This project reproduces the aggressive, unfiltered proactive-scaling trade-off NimbusGuard reports, then applies exactly this kind of smoothing/hysteresis rule on top of the same underlying predictive signal, to test whether it recovers stability without giving back the SLA benefit of forecasting in the first place.

3 Methodology

3.1 Synthetic Workload

A synthetic cyclical workload (`src/data/workload_simulator.py`) of 500 timesteps combines a sinusoidal base load, periodic traffic spikes (every 80 steps), and Gaussian noise – generating the kind of bursty, partially-predictable demand pattern proactive scaling is meant to handle better than reactive thresholds.

3.2 Training and Evaluation

For each of 5 seeds (42-46), a fresh workload is generated and a fresh predictor is trained on it. All three scaling policies (Reactive HPA, Aggressive PAKS, Stability-Aware PAKS) are then run over the same workload and evaluated identically, so any difference in outcome reflects the scaling policy, not the underlying data or model.

3.3 Metrics

- **SLA Violations:** count of timesteps where allocated pods fall below the absolute minimum required to serve the actual workload without throttling.
- **Over-provisioning %:** total allocated pod-steps in excess of the minimum required, as a percentage.
- **Scaling Events:** count of timesteps where the pod count changes from the previous timestep – directly comparable to NimbusGuard’s own “Total Scaling Events” metric.
- **Pod Count Volatility:** standard deviation of step-to-step pod-count changes – a finer-grained stability measure than a raw event count.

All code and results are in `paks-framework/`.

4 Design Specification

4.1 Reactive HPA

Scales strictly on the currently-observed workload, with a one-step reaction delay – the industry-standard reactive baseline NimbusGuard itself compares against.

4.2 Aggressive PAKS (baseline reproduction)

A feed-forward neural network (scikit-learn `MLPRegressor`, 2 hidden layers) is trained to predict the next-step workload from the previous 10 steps. At every step, the predicted load (with a small 5% safety buffer) is converted directly into a target pod count and applied immediately – no filtering. This reproduces the core behaviour NimbusGuard’s DQN+LSTM agent exhibits: proactive, but reacting to every raw forecast, including noise.

4.3 Stability-Aware PAKS (proposed improvement)

Built on the exact same predictor as Aggressive PAKS, wrapped in a `StabilityAwareController` that adds two changes before a scaling action is taken:

- **Exponential smoothing** of the raw predictions ($\alpha = 0.4$), so a single noisy forecast cannot on its own trigger a scaling action.
- **Hysteresis + cooldown**: a scaling action only fires if the smoothed prediction implies a pod count that differs from the current one by more than 1 pod, *and* at least 2 steps have passed since the last scaling action.

This is a direct, minimal implementation of the calmer design NimbusGuard’s own future-work section gestures at (“expanding the action space,” hybridising with more conservative scaling), applied as a simple rule rather than a second learned agent.

5 Implementation

The system is implemented in Python 3 with scikit-learn. Key modules:

- `src/data/workload_simulator.py` – synthetic cyclical workload generator described in Section 3.
- `src/models/scalers.py` – `run_reactive_hpa`, `run_aggressive_paks` (baseline), and `StabilityAwareController / run_stability_aware_paks` (improvement), sharing one `train_predictor` function.
- `scripts/train_and_evaluate.py` – trains and evaluates all 3 policies across 5 seeds, writes `results/results_per_seed.csv` and `results/results_summary.csv`, and exports the trained predictor to `src/lambda_handler/model/` for deployment.
- `src/lambda_handler/app.py` – an AWS Lambda handler that applies the same stability-aware controller logic to live workload telemetry arriving on a Kinesis stream, one instance of the controller per node, persisted across warm invocations.
- `template.yaml` – AWS SAM template provisioning the Kinesis stream and Lambda function. This replaces an earlier scaffold template that pointed `Handler` at a `src/ml_predict.py` file that did not exist anywhere in the repository, and referenced hardcoded placeholder VPC subnet IDs that would have failed on any real deploy attempt.

- `.github/workflows/deploy.yml` (repo root) – CI pipeline that runs the training/evaluation script and, on push to `main`, deploys the SAM stack to the configured AWS region using repository secrets.

The original scaffold used TensorFlow/Keras for the workload predictor; this version uses scikit-learn’s `MLPRegressor` instead (same 2-hidden-layer feed-forward design), removing a heavy dependency the project did not otherwise need. Running `python scripts/train_and_evaluate.py` reproduces every number in Section 6 from scratch. The Lambda handler was verified locally against a synthetic rising-workload sequence before deployment.

6 Evaluation

Table 1 reports mean $\pm$ standard deviation across 5 seeds (42-46).

Table 1: Results across 5 seeds (mean $\pm$ std)

Model	SLA Violations	Over-prov. %	Scaling Events	Pod Volatility
Reactive HPA	20.0 $\pm$ 2.7	40.92 $\pm$ 0.19	407.4 $\pm$ 4.9	3.32 $\pm$ 0.25
Aggressive PAKS (baseline)	11.0 $\pm$ 2.2	45.80 $\pm$ 2.23	384.4 $\pm$ 9.1	2.86 $\pm$ 0.36
Stability-Aware PAKS (improved)	18.0 $\pm$ 2.7	46.52 $\pm$ 2.16	165.8 $\pm$ 5.3	1.95 $\pm$ 0.17

6.1 Experiment 1: Reproducing NimbusGuard’s Trade-off

Aggressive PAKS reproduces the paper’s central claim: proactive scaling roughly halves SLA violations relative to reactive HPA (11.0 vs. 20.0), at the cost of higher over-provisioning (45.8% vs. 40.9%). Scaling-event counts are comparable between the two here (384.4 vs. 407.4) rather than showing NimbusGuard’s reported 2x gap – a difference in the specific numbers that is expected, since this is a much simpler feed-forward predictor on a synthetic workload rather than a DQN+LSTM agent on a real Kubernetes testbed, but the qualitative trade-off (better SLA, worse over-provisioning) reproduces cleanly.

6.2 Experiment 2: Stability Recovered

Stability-Aware PAKS is where the improvement shows up clearly: scaling events drop by 57% (384.4 $\rightarrow$ 165.8) and pod-count volatility drops by 32% (2.86 $\rightarrow$ 1.95) relative to Aggressive PAKS – both well outside one standard deviation, and the specific instability NimbusGuard’s own results table names as its weakness.

6.3 Experiment 3: The Honest Cost

The stability fix is not free. SLA violations for Stability-Aware PAKS (18.0) are higher than Aggressive PAKS (11.0) – smoothing and cooldown necessarily mean reacting slightly later to genuine spikes, not just to noise. It still beats reactive HPA (20.0), but by a much smaller margin than the unfiltered aggressive policy does. Over-provisioning is essentially unchanged from Aggressive PAKS (46.5% vs. 45.8%) – the stability fix targets scaling frequency and volatility specifically, not resource cost.

6.4 Discussion

This is a genuine trade-off, not a strictly dominant improvement, and it is reported as such rather than only highlighting the metrics that favour the new design. The choice between Aggressive and Stability-Aware PAKS in a real deployment would depend on which is more expensive for a given operator: a handful of extra SLA violations, or a cluster that reconfigures itself 165 times instead of 384 over the same window. Section 7 discusses how the smoothing/cooldown parameters could be tuned to move along this trade-off rather than treating the reported configuration as the only point on it.

7 Conclusion and Future Work

This project asked whether a proactive Kubernetes autoscaler could keep most of its SLA-violation advantage over reactive HPA while substantially reducing the instability that Wanigasooriya & Ekanayake’s (2026) NimbusGuard exhibits and names but does not fix. Across 5 seeds, a stability-aware controller (exponential smoothing + hysteresis/cooldown, applied to the same forecasting model as an unfiltered baseline) cut scaling events by 57% and pod-count volatility by 32%, while still beating reactive HPA on SLA violations – though giving back roughly half of the SLA-violation advantage the unfiltered proactive policy achieves.

The honest finding is a trade-off surface, not a single winning configuration: the smoothing factor and hysteresis/cooldown thresholds used here (0.4, 1 pod, 2 steps) are one point on a continuum between “aggressive and unstable” and “calm but slower to react,” and were not exhaustively tuned.

Future work:

- Sweep the smoothing/hysteresis/cooldown parameters to map the full SLA-vs-stability trade-off curve, rather than reporting one configuration.
- Evaluate on a real Kubernetes testbed with real workload traces, as NimbusGuard itself does, rather than a synthetic generator.
- Explore whether the cooldown/hysteresis rule could itself be learned (e.g. as a second, small model deciding *when* to trust the predictor) rather than fixed, closing the gap to NimbusGuard’s own RL-based approach while keeping the simplicity of not requiring a full DQN pipeline.

References

Wanigasooriya, C. and Ekanayake, I. (2026). Nimbusguard: A novel framework for proactive Kubernetes autoscaling using deep Q-networks, *2026 40th International Conference on Information Networking (ICOIN)*, IEEE. Preprint: <https://arxiv.org/abs/2604.11017>.